Master Réseaux et Systèmes Distribués USTO-MB
M.N OSMANI

TP 0 : Préparation de l'environnement de développement (Windows & VS Code)

L'objectif de cette étape préliminaire est d'installer la chaîne de compilation C (**GCC / MinGW-w64**) et de configurer **Visual Studio Code** pour pouvoir compiler et exécuter des programmes réseau.

1. Installation de MinGW-w64 (Compilateur GCC)

Pour compiler du code C sous Windows avec l'API réseau Windows (Winsock), nous utilisons l'environnement classique **MINGW64** de MSYS2 (évitant les incompatibilités d'exécution).

- Rendez-vous sur le site officiel de MSYS2 : https://www.msys2.org/.
- éléchargez le fichier d'installation exécutable le plus récent nommé : msys2-x86_64-latest.exe.
- Installer msys2-x86_64-latest.exe.
- **Dossier d'installation :** Laissez le chemin par défaut recommandé : C:\msys64.
- Ouvrez l'application **MSYS2 MINGW64** depuis le menu Démarrer de Windows.
- Mettez à jour le gestionnaire de paquets et installez la chaîne GCC en exécutant :

```
pacman -S --noconfirm mingw-w64-x86_64-gcc
```

- **Vérification :** Tapez `gcc --version` dans le terminal MSYS2. Le numéro de version de GCC doit s'afficher.

2. Configuration des Variables d'Environnement Windows (PATH)

Pour pouvoir appeler la commande `gcc` depuis n'importe quel terminal :

- Appuyez sur le raccourci clavier Win + R, saisissez `sysdm.cpl` puis appuyez sur **Entrée**.
- Allez dans l'onglet **Paramètres système avancés**, puis cliquez sur le bouton **Variables d'environnement...**.
- Dans le cadre inférieur (**Variables système**), sélectionnez la variable **Path** et cliquez sur **Modifier...**.
- Cliquez sur **Nouveau** et ajoutez le chemin exact :

```
C:\msys64\mingw64\bin
```

- Si une ligne contenante `C:\msys64\ucrt64\bin` existe déjà, supprimez-la ou remplacez-la par le chemin ci-dessus pour éviter les conflits de bibliothèques dynamiques.

- Cliquez sur **OK** sur toutes les fenêtres pour valider l'enregistrement.

3. Configuration de Visual Studio Code

- Pour s'assurer que le terminal interne de VS Code détecte automatiquement GCC :
- Démarrez **VS Code**.
- Ouvrez les paramètres au format JSON : appuyez sur Ctrl + Maj + P, tapez Preferences: Open User Settings (JSON) et validez.
- Vérifiez ou ajustez les sections suivantes dans votre fichier de configuration pour utiliser un terminal standard et injecter le chemin :

```
{
  "terminal.integrated.defaultProfile.windows": "Command Prompt",
  "terminal.integrated.env.windows": {
    "PATH": "C:\\msys64\\mingw64\\bin;${env:PATH}"
  }
}
```

- Enregistrez les modifications.
- Fermez complètement VS Code (**Fichier > Quitter**) puis rouvrez-le.

4. Validation finale de l'outillage

- Ouvrez un terminal dans VS Code (Terminal > Nouveau terminal).
- Exécutez la commande de contrôle :

```
gcc --version
```

- Résultat attendu :

Le terminal doit renvoyer la version de MinGW-w64 (par exemple gcc.exe (Rev..., Built by MSYS2 project)...) sans afficher d'erreur du type *« 'gcc' n'est pas reconnu... »* ou d'alerte de DLL manquante.

5. Règle de compilation pour les TPs réseau sous Windows

Contrairement aux programmes C classiques, tout programme utilisant les sockets sous Windows nécessite l'édition de liens avec la bibliothèque système `ws2_32`.

La commande de compilation se présentera systématiquement sous cette forme :

```
gcc nom_du_fichier.c -o nom_du_fichier.exe -lws2_32
```